

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 06 — INTRODUCTION TO MODBUS

Student Edition

Student name	Sidra Shaikh
Registration No.	C11/26/ACIS/17349
Date	30 September 2026

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 06: Introduction to Modbus

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 3–4 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Explain the difference between Modbus TCP and Modbus RTU.
- Identify common Modbus function codes and data areas.
- Observe Modbus TCP communication between FUXA and OpenPLC in an isolated lab.
- Use Wireshark to capture/filter Modbus traffic and identify request/response fields.
- Relate register/address information to the running tank-fill process.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password
SCADA VM	192.168.2.10	scada	password
Analyst/Kali VM	Instructor-provided	kali	kali

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Reference: Modbus Function Codes

Code	Function	Purpose
01	Read Coils	Read discrete output/coils
02	Read Discrete Inputs	Read discrete input states
03	Read Holding Registers	Read 16-bit holding registers
04	Read Input Registers	Read 16-bit input registers
05	Write Single Coil	Change one discrete coil
06	Write Single Holding Register	Change one holding register
15	Write Multiple Coils	Change multiple coils
16	Write Multiple Holding Registers	Change multiple registers

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start the PLC, SCADA and analyst VMs on the instructor-provided isolated laboratory topology. Do not connect this lab to production or bridged networks.

Step 2. On the PLC VM, start the TankFillSim program in OpenPLC Runtime and confirm Running status.

Step 3. On the SCADA VM, start FUXA, load the tank-fill HMI and confirm the openplc connection targets 192.168.1.10:502.

Step 4. Run the HMI and verify that the tank level changes between its operating limits.

Step 5. On the analyst VM, open Wireshark. Select only the interface connected to the ICDFA lab segment carrying PLC/SCADA traffic.

Step 6. Apply the display filter modbus. If needed, refine it with `tcp.port == 502`.

Step 7. Start a short packet capture while the HMI is running, allow several process updates to occur, then stop the capture.

Step 8. Select a Modbus request/response pair and expand the Modbus/TCP and Modbus protocol fields.

Step 9. Identify the transaction/request, source and destination IP addresses, function code and referenced register/data value.

Step 10. Locate a Function Code 4 (Read Input Registers) response associated with the tank-level value. In the teaching image, Input Register 80 corresponds to the Tank Level indication.

Step 11. Complete the Modbus function-code table by explaining at least codes 01, 02, 03, 04, 05, 06, 15 and 16.

Step 12. Save the packet capture with a filename containing your registration number and LAB06. Do not capture unrelated traffic.

Required Evidence & Submission

- Screenshot of the FUXA/OpenPLC process running.
- Screenshot of Wireshark filtered to Modbus traffic.
- One annotated request/response analysis identifying function code and register.
- The saved .pcap/.pcapng capture from the isolated training network.
- A completed function-code table and a short comparison of Modbus TCP versus Modbus RTU.

Recommended filename: <RegistrationNo>_LAB06_Introduction_to_Modbus.pdf plus the required project/capture file(s).

Knowledge Check

1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Response: _____

2. Why does Modbus use function codes?

Response: _____

3. What does Function Code 4 request/read?

Response: _____

4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Response: _____

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.

Solution Below



1. Objective

This lab observes live Modbus TCP communication between the FUXA HMI and OpenPLC Runtime, and uses Wireshark to capture, filter and analyse that traffic, identifying request/response pairs, function codes, and the register values corresponding to the running tank-fill process.

2. Procedure Summary

The PLC and SCADA VMs from Lab 05 were reused: OpenPLC Runtime was confirmed Running with the Tank Fill Sim program, and FUXA's HMI was confirmed connected and animating, polling TankLevel_LIT (Input Register 80) and TankValveOpen (Coil 0) over Modbus TCP to 192.168.1.10:502.

Rather than capturing from an endpoint's own interface, a separate Analyst VM was built specifically for this lab (Debian, with Wireshark), attached to the same isolated internal network as the PLC and SCADA VMs, with Promiscuous Mode enabled so it could passively observe traffic between the other two machines as an independent, unauthorized third party would. In Wireshark, the modbus display filter was applied on the internal-network interface, and a capture was taken while the HMI ran through several fill/drain cycles.

The resulting capture clearly shows repeating Function Code 1 (Read Coils) and Function Code 4 (Read Input Registers) query/response pairs on a roughly 10ms scan cycle, all in plaintext with no authentication.

3. Required Evidence

3.1 PLC/FUXA process running

OpenPLC Runtime's Dashboard confirming the Tank Fill Sim program Running, and the FUXA HMI showing the tank level actively animating, providing the live process data that was subsequently captured.

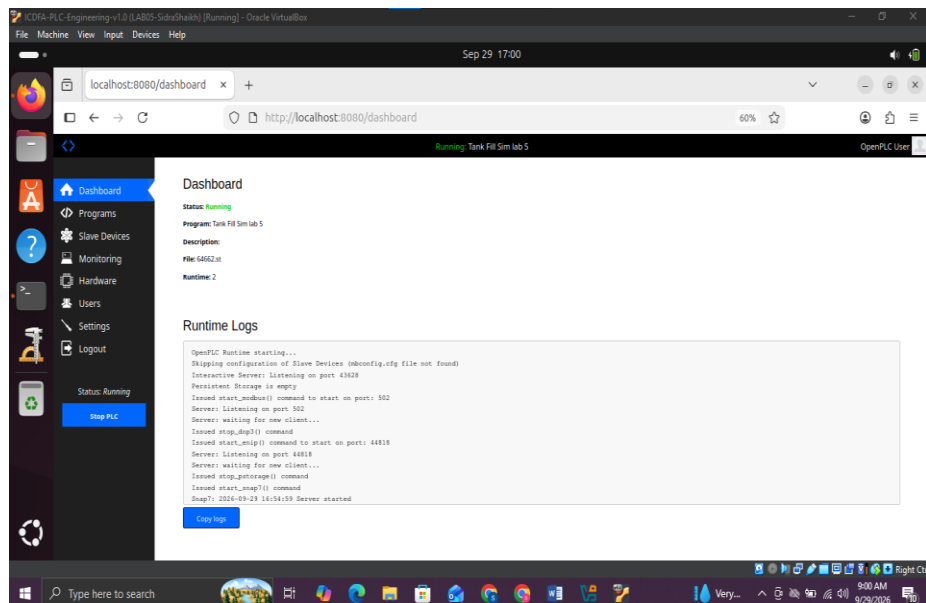


Figure 1. OpenPLC Runtime Dashboard — Tank Fill Sim running.

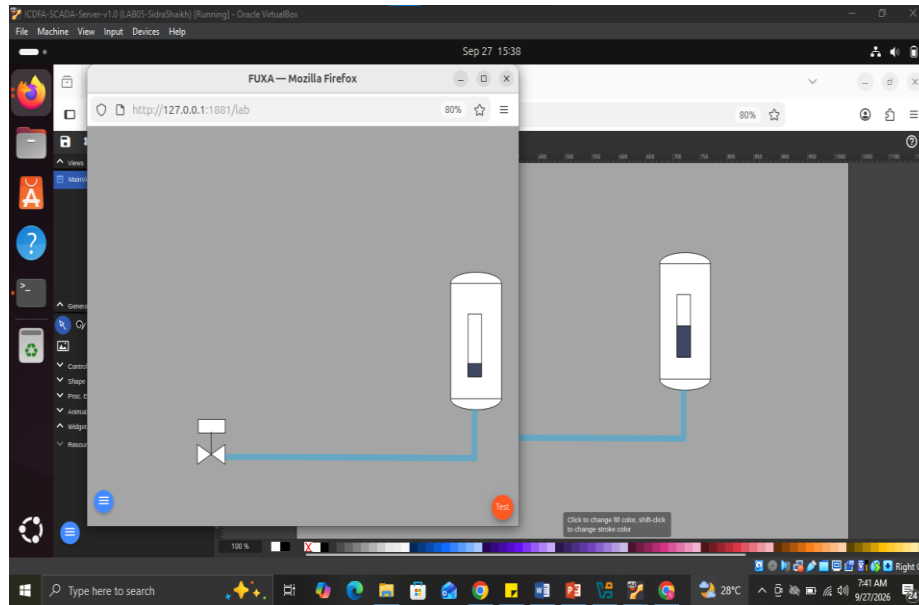


Figure 2. FUXA HMI showing the tank-fill process live.

3.2 Wireshark filtered to Modbus traffic

A live capture on the Analyst VM's internal-network interface (enp0s3), filtered with modbus, showing repeating Read Coils (Function 1) and Read Input Registers (Function 4) query/response pairs between the SCADA VM (192.168.1.20) and the PLC VM (192.168.1.10), captured passively from a third machine on the network.

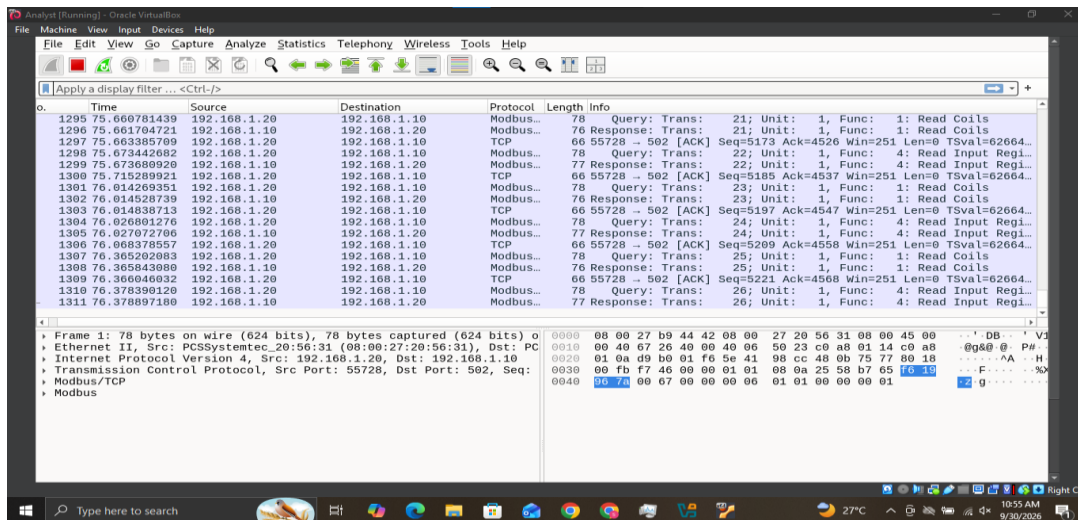


Figure 3. Wireshark capture filtered to Modbus traffic between SCADA and PLC.

3.3 Annotated request/response analysis

Packet 1305 is a Function Code 4 (Read Input Registers) response from the PLC (192.168.1.10) to the SCADA VM (192.168.1.20). The expanded Modbus/TCP layer shows Transaction Identifier 24, Protocol Identifier 0, Length 5, Unit Identifier 1. The expanded Modbus layer shows Function Code: Read Input Registers (4), Byte Count 2, and the returned value: Register 80 (UINT16) = 44 — the live Tank Level reading at the moment of capture, transmitted in plaintext and readable to any device with access to this network segment.

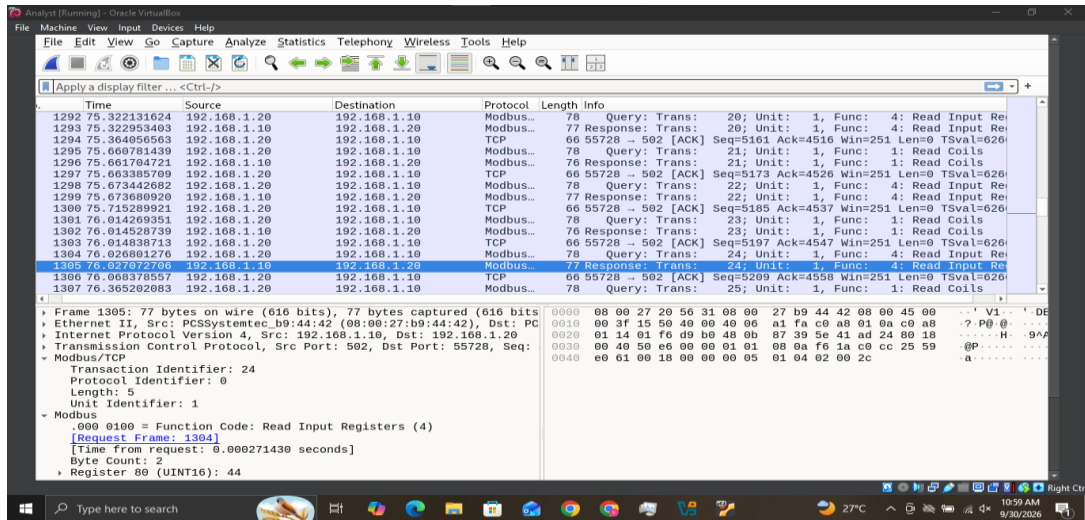


Figure 4. Expanded Function Code 4 response — Register 80 (Tank Level) = 44.

3.4 Packet capture file

Capture saved as C11-26-ACIS-17349_LAB06_capture.pcapng, submitted alongside this report.

https://github.com/SidraShaikh-2/ICDFA-Assignments/tree/main/ACICP301%20%E2%80%94%20ICS%20%26%20SCADA%20Systems/C11-26-ACIS-17349_LAB06_capture_ProjectFiles

4. Modbus Function Code Reference

Code	Function	Explanation
01	Read Coils	Reads the ON/OFF status of discrete output points (coils). Used here to read TankValveOpen.
02	Read Discrete Inputs	Reads the ON/OFF status of discrete input points, separate from coils; typically wired sensor/switch inputs.
03	Read Holding Registers	Reads 16-bit read/write registers, commonly used for setpoints or general-purpose analog values.
04	Read Input Registers	Reads 16-bit read-only registers, typically live measured process values. Used here to read TankLevel_LIT at Input Register 80.
05	Write Single Coil	Writes a single discrete output (coil) to ON or OFF, e.g. commanding a valve open or closed.
06	Write Single Holding Register	Writes a single 16-bit value into one holding register, e.g. changing one setpoint.
15	Write Multiple Coils	Writes ON/OFF values to a contiguous block of coils in one request, rather than one at a time.

16	Write Multiple Holding Registers	Writes values into a contiguous block of holding registers in one request, rather than one at a time.
----	----------------------------------	---

5. Modbus TCP vs. Modbus RTU

Modbus RTU is a serial protocol, running over RS-232/RS-485 connections with binary-encoded messages and a CRC checksum, typically used point-to-point or on a shared multi-drop serial bus with devices distinguished by a unit/slave address. Modbus TCP carries the same underlying Modbus data model and function codes, but encapsulates them inside a TCP/IP packet (adding an MBAP header with a transaction identifier, as seen in Figure 4), and runs over standard Ethernet/IP networks on port 502, using the destination IP address rather than serial wiring to reach a specific device.

In this lab, TCP's transaction identifier (e.g. 24 in Figure 4) serves a role serial RTU does not need, since RTU has only one conversation on the wire at a time; TCP allows multiple in-flight requests to be matched to their responses. Practically, Modbus TCP's use of ordinary IP networking is also what made this lab's third-party capture possible: unlike a private serial bus, anything sharing the same Ethernet segment, including an unauthorized Analyst VM with promiscuous mode enabled, can observe every request and response in plaintext.

6. Knowledge Check

1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Modbus RTU runs over a serial connection (RS-232/RS-485) with binary framing and a CRC checksum, addressing devices by a unit/slave ID on a shared serial bus. Modbus TCP instead runs over standard Ethernet/IP networks on TCP port 502, wrapping the same Modbus data in a TCP/IP packet with an added MBAP header (including a transaction identifier), and addressing devices by IP address rather than serial wiring.

2. Why does Modbus use function codes?

Function codes tell the receiving device what kind of operation is being requested, for example reading coils, reading input registers, or writing a holding register, so that a single simple protocol can support many different read and write operations against different data areas. The function code, together with a register/coil address, is what lets a generic Modbus message represent a specific, meaningful instruction like 'read the value at Input Register 80.'

3. What does Function Code 4 request/read?

Function Code 4 requests a read of one or more Input Registers, 16-bit read-only values typically used for live measured process data. In this lab, Function Code 4 was used to read Input Register 80, which holds the simulated Tank Level value (see Figure 4, where the response returned a value of 44).

4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

As this lab demonstrated directly, Modbus carries no encryption and no authentication: a third device with no credentials and no relationship to either endpoint (the Analyst VM) could passively observe every request, function code and data value passing between the SCADA system and the PLC, including the live Tank Level reading. In a real deployment, the same visibility would expose an attacker to process setpoints, valve states and other operational data, and the same lack of authentication would in principle allow a device with network access to also send its own Modbus write requests (Function Codes 5, 6, 15, 16), directly manipulating the process rather than just observing it. This is why ICS network segmentation, monitoring and access control matter far more than they would for an already-encrypted protocol.